QCon
全球软件开发大会

InfoQ 技术大会

AI 时代的移动端稳定性保障

柯南 AI Agent 实践

王泽锋 · QCon 上海 2026



自我介绍



王泽锋

快手 / 主站技术 · 大前端稳定性团队

2018 毕业入行，长期从事移动端开发

早期 · 腾讯 Android / iOS 双端客户端 SDK、小程序框架

2023 · 快手 Android / iOS / Harmony 三端稳定性全链路建设

最近半年多 专注 AI + 移动端稳定性的结合 —— 柯南 AI

目录

01 新挑战 —— AI 时代的稳定性命题

02 Agent First 的稳定性新工作范式

03 鸿蒙 OOM 治理实战

04 总结与展望

01 新挑战

节奏错配 × 全新战场

现状：上下游速率不对等

上游：研发侧吞吐

AI Coding 已成主流，需求到代码
的转化成本被压到极低

转化成本 ↓ 一个量级

下游：稳定性处置

线上告警、根因定位、修复回归——
每个环节本质上还是人在扛

处置成本 ≈ 零加速

上游生产到下游消费的速率不对等，
长期看线上有稳定性全面劣化的风险

局部最优陷阱：AI 自身还在制造新问题

缺乏全局上下文，AI 倾向沿「最小阻力路径」让问题**现象消失**，而不是真正解决问题

稳定性问题恰恰大量来自「全局副作用」——跨工程、跨模块、跨版本、跨运行时，公域语料里学不到完整心智模型

局部正确

全局有副作用

这一处改对了，触发
另一模块的隐式依赖

跨边界场景

有坑

单端跑通，多端 / 多语言
/ 多版本组合下失效

这类问题正随 AI 生码规模扩大被**批量产生**

前置拦截不可能消灭所有问题

SDD、Coding Harness 等前置手段能让产出更优，但大前端的特点是**线上状态空间爆炸**

100+ ROM
1000+ 机型

Android 机型碎片化
线上状态的第一层组合

上万次 / 天

强运营 App 的开关、AB、
活动等变更动作组合

千人千面

用户行为路径无法穷举
线下测试不可能完全覆盖

软件稳定性的本质是不完备性证明问题

永远无法证明所有情况都稳定，只能持续增加防御深度

线上真实现场，永远是稳定性的核心战场

第二战场：历史债 × 一码三端

鸿蒙防御工事未建好，需求与 AI 生码已赶上

1

鸿蒙成为移动端第三极

- 多平台并行新阶段：Android / iOS / 鸿蒙
- 不是投不投，而是快不快

2

鸿蒙原生技术栈背着历史债

- 需求持续高速堆积
- 基建欠双端多年工具链

3

效能约束推大跨端 AI 再踩一脚油门

- 三端人效推大 KMP 跨端
- FFI / 跨语言内存模型是公域知识最稀薄处

一个背着历史债、问题天然跨栈、又被 AI 踩了脚油门的战场

小结：新挑战

研发吞吐
被 AI 拉快
→
处置后链路
没有跟上
→
AI 自身有
局部最优陷阱
→
前置拦截不可能
消灭所有问题

当生产吞吐被 AI 拉开一个量级，处置吞吐必须有
一种不依赖人头线性增长的方式跟上——
这个方式，只能是 Agent 自己。

第二战场：鸿蒙历史债 × AI 时代一码三端——基建最薄弱、问题最跨边界、没有作业可抄、却必须打

AI 让存量战场节奏变快，历史债 × 一码三端让增量战场难度变大——
稳定性保障，需要一次范式级的升级。

02 新范式

Agent First

柯南 AI 是什么



真相只有一个

定位：**快手移动端的稳定性数字同事**

100% 大前端业务场景覆盖	76% 用户渗透率

~80% 结论采纳率	~30% 三端指标治理降幅	~30% 报警处置 MTTR 降低

用户评价



线上稳定性处置最重要的两件事

发版前

发版必解

灰度阶段发现、必须正式发版前修复的问题

把问题上线前的**最后一道闸门**

发版后

报警处置

线上出事了怎么办——
从告警触发到止损、定位、复盘

完整处置链路的**时效与质量**

报警处置 case：处置流程

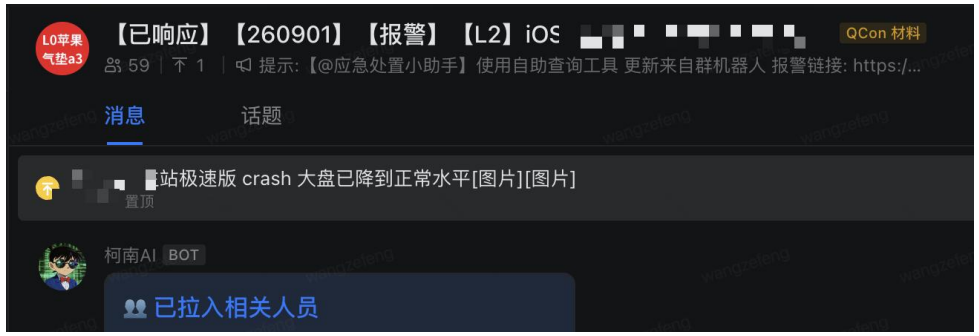
过去：大量时间花在「信息聚合」和「人对齐」上。现在：柯南接手——以下为一次真实报警的处置流程



① 报警发生



② 柯南拉群



③ 柯南拉人



④ 周期性同步进展

从告警触发到处置推进，**信息聚合与人对齐全部由柯南完成**

报警处置 case：过程中的分析产出



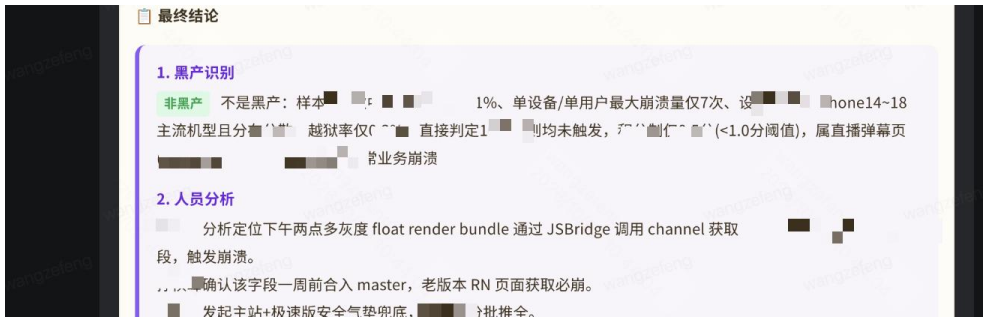
① 聚集特征分析 版本 / 设备 / 堆栈聚合



② 临时止损建议 自动创建气垫



③ 人工确认止损 值班同学一键确认



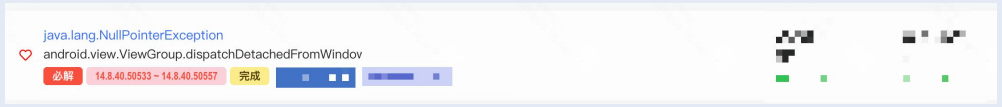
④ 报警总结 沉淀进知识库

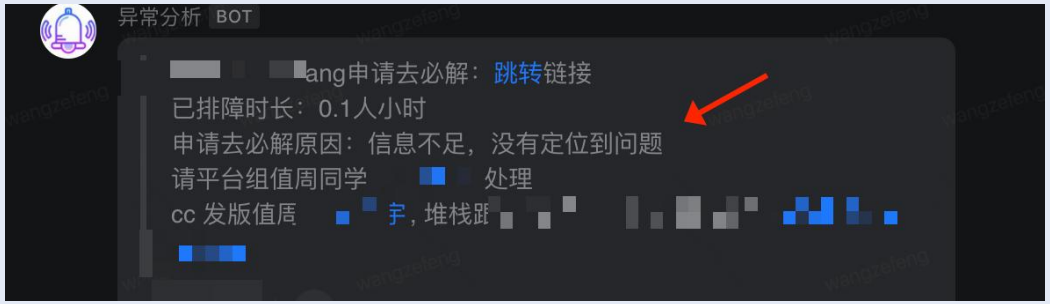
值班同学从「情报员 + 调度员 + 决策员」变成「确认者」

发版必解 case

过去：信息不足 → 去必解







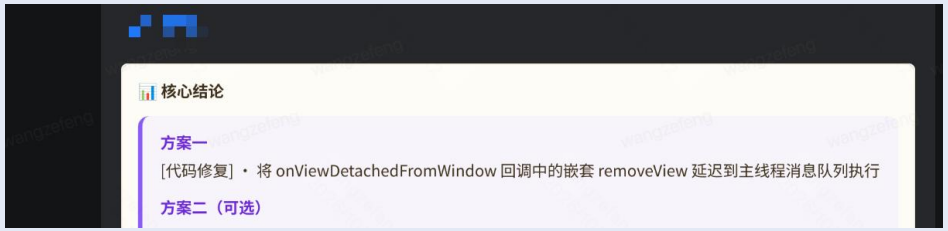
堆栈全是系统帧、无一行业务符号，只在新版本出现

业务方判断「无法定位」→ 提交「去必解」申请

现在：去必解自动分析 → 当天修复







自动触发深度分析：根因 → 证据链 → 修复建议，业务同学当天合入修复

从「去必解 → 放弃」到「定位 → 当天修复」——**最大增益来自「原本会被放弃的那批疑难问题」**

Model 与 Agent 的判断

模型 = 新入职的行业巨佬

长处：技术顶尖 —— 超强推理、公域知识顶尖

短处：

① 不懂公司黑话 —— 内部平台、私域经验一概不知

② OKR 不清楚 —— 目标 / 停点没人给

→ 教私域 / 喂现场 / 给停点

确定性的事交给脚本：又快又准

Agent

Coding Agent 是终极解法 —— 能力最成熟、持续重投入

选择：不造轮子，站在肩膀上

Harness 尽量薄：投入在补环境

轻 Agent，重上下文建设

柯南的 Agent 架构


1
产品层
嵌入现有工作流
自动工作流：报警 / 必解
主动触发：IM / 监控看板
...
2
编排层
确定性的用代码；
单 LLM 优先于 Agent
Workflow（多 Agent）
eg. 报警工作流
单 LLM
eg. 报警聚合分析
Agent
eg. 根因分析
3
Agent Harness 层
侧重 Skills 建设（上下文）
其余做薄
System Prompt / Tools / Permissions / ...
Skills
原子能力 / 结束点 / 私域知识
4
数据层
Agent 的感知输入
日志
源码服务
产物服务
变更
...
5
端侧基建层
决定能采集到什么样的现场
堆栈
内存图谱
Trace
...
反馈迭代 · 可观测

柯南的 Agent 架构 · 迭代反馈



迭代反馈 · 难在哪里

重要性是共识——反馈是 Agent 持续变好的唯一来源。真正值得讲的是它难在哪：

困难一（Agent 本身）：技术成功 ≠ 任务成功

会话跑完、卡片发出、日志全绿 = 技术成功；用户发现结论方向是反的 = 任务失败

接口成功率 / 工具调用成功率 / P95 延迟，回答不了「答得对不对」

困难二（线上稳定性场景）：额外的两层

① **用户不一定有评判能力**：似是而非的结论，业务同学缺乏稳定性专家视角，可能照单全收

② **结果不一定直接可验证**： 流程长；结果正确 ≠ 过程正确——跳步碰巧对，不可持续

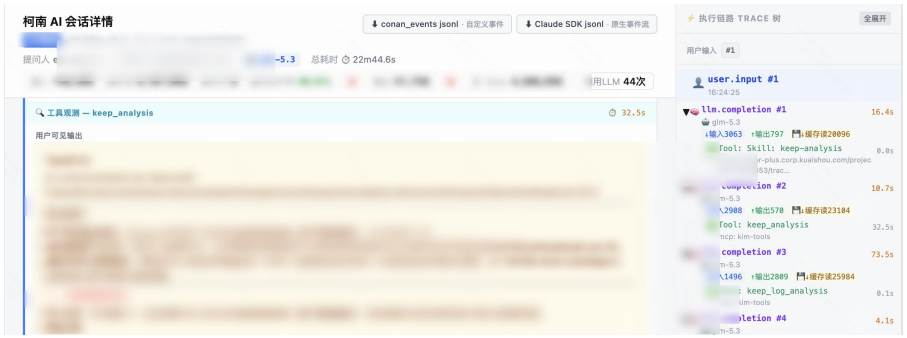
从持续提升效果的视角，推理过程的正确性必须被审视

必须有人审，而且必须是专家审

迭代反馈 · 可观测基建

① Session 全量观测

jsonl 是 Coding Agent 观测的事实标准；在其上打补丁——
适配 workflow / 多 Agent / 单 LLM，串回单点全流程



② Case 标注平台

筛选 → 打标 → 归因 → 沉淀优化项



像分析 coredump 一样分析 Agent 的失败 case——没有这一层，「为什么不对」只能靠猜
Agent 的失败变成可回放、可定位的工程问题

迭代反馈 · Review 机制：人审 + AI 审

人审：每周 Case Review 会议

固定会议保证机制真正落实、保证 review 质量

雷打不动 · 每周逐条过

发现即沉淀优化项



AI 审：独立审计 Agent review 历史 session

人工看一条完整会话成本极高，上限直接决定迭代速度

维度一 · 分析效率：绕路归因到五类可改项
（数据字段缺失 / 工具输出误导…）

维度二 · 结论置信度：每条「已确认」论点回溯证据，
扫描五类逻辑漏洞（循环论证 / 假设已证伪未降级…）

打标必须引用 thinking 原文；扣分必须给出证据位置
只评价「路径是否高效、推理是否严密」

定位：人审把关、AI 审提效——AI 审产出的不是判定，是给人审的候选

迭代反馈 · 反馈闭环：发现落到哪一层

落到哪一层	Review 发现	修复动作
产品层	用户对产品形态与交互的反馈	调整产品形态与交互设计
Harness 层	私域知识缺失	进知识库，或沉淀为 Skill
数据层	某工具调用失败率高	优化接口与友好报错
端侧基建	出现新的现场数据诉求	反向驱动采集方案补齐

不是「先规划好一套基建」，而是「线上 case 告诉我们哪里不够，就把哪里补上」

柯南的 Agent 架构 · Skills



Skills · 根因分析 Skills：主体不是 SOP

核心观点：根因分析 Skills 的主体不是 SOP —— 推理是「巨佬」的强项，不试图教大佬做事
早期踩坑：事无巨细的 SOP，规定得越死，Agent 越笨

组成部分一 · 原子能力

给 Agent 提供手和脚

组成部分二 · 结束点

和模型对齐什么时候停

去掉事无巨细的 SOP 之后，Skills 最主要的就是两个组成部分：原子能力 + 结束点

Skills · 结束点落地

结束点的关键在于清晰定义——不加干预，模型的根因结论会出什么问题?

「看起来像是对的」陷阱

最典型的失败不是「查不出来」，是「编一个像的」—— 结构完整、逻辑自洽、格式漂亮

每一样都对，唯独结论不成立：逻辑自洽 ≠ 正确；格式正确性，是 AI 内容最危险的伪装

我们把它落成两个机制——

机制一 · 取证

证据不够不许停

→

机制二 · 定级与表达

结论分层，推测不许冒充事实

Skills · 机制一（取证）：把「再想想」变成确定动作

「仔细检查后再回答」是祈祷不是机制——柯南 Crash Skill 把自评固化成四个问题，**每个绑定一个不通过时的强制动作**

Q1 堆栈解释

能否 1-2 句话说清这个崩溃大概是怎么回事?

不通过 → 拉取崩溃帧源码

Q2 本质原因

能否定性「只要什么条件成立，这里就必然崩溃」?

不通过 → 沿调用链向上追溯

Q3 直接原因

条件具体是什么? 新引入问题的变更 commit 是什么?

不通过 → 强制逐帧 blame

Q4 事实锚定（灵魂）

是否有来自真实崩溃样本的正向证据，确认条件确实成立?

不通过 → 扫日志找佐证

「时间上吻合」≠「因果上成立」——「再想想」被替换成「去 blame」「去扫日志」这样**可检查的动作**

Skills · 机制二（定级与表达）：事实与推测分层

报告必须严格分层，混层即判为错误报告——核心一条：事实与推测必须分开

L1–L3 事实层（与 Q1–Q3 一一对应）

有充分证据才填，无证据不得虚填；每条结论附关键证据来源

三层分别回答：怎么回事（L1）/ 为什么（L2）/ 哪次变更引入（L3）

L4 推测方向（必须标注「推测」）

Q4 不通过时结论降级到这里：方向 + 现有依据 + 缺失什么才能确认

兜底写法：宁可写「深层原因待进一步验证，缺少 ××」，不得强行编造

「我不知道」是被允许的、格式正确的答案——短期是体验损失，长期是信任的本金

Skills · 私域知识：知识库为主体，SOP 做快速路径

私域知识的主体是知识库（按需查询）；共性典型问题、有更快定位路径的，才进垂类 SOP

通道一 · 知识库

内部机制原理 / 平台用法与口径，遇到才查

通道二 · 垂类 SOP

共性典型问题、能整合出更快定位路径的，才沉淀进 Skill

快速路径 · 垂类 SOP | 例：插件化 NoClassDefFoundError

教科书答案（搜源码、查混淆）在插件化框架下经常全错

相当比例根因是插件未安装 / 边界被破坏

SOP 显式写一句「优先排查插件归属」，直接走对分支



经济账：不问「模型能不能做」，问「值不值得用推理去做」——每条快速路径 = 一次推理换成一次确定性查表

柯南的 Agent 架构 · 数据基建



数据基建 · 应接尽接 + 脚本化批量

数据提供范围

应接尽接

对排障有用的数据源，就接给 Agent：

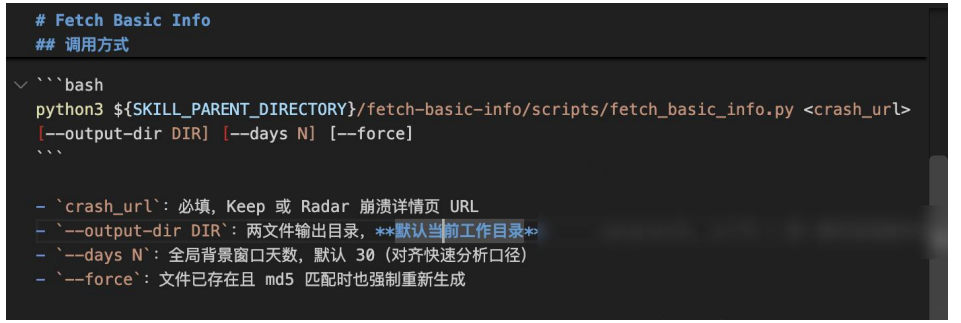
- 基础崩溃数据
- 源码服务
- 日志包下载
- 产物下载
- 开关与实验变更
- 堆栈符号化
- 气垫创建
- ……

数据提供方式

脚本化 + 适当批量

脚本化：一个能力 = 一个命令行脚本——确定性的事交给脚本
参数 / 结构化输出 / 明确退出码，一次调用拿到全部

适当批量：一次调用拿全，减少 tool call 与思考轮次——
fetch-basic-info：一个崩溃 URL 拉齐符号化堆栈、
影响量级、版本集中度等 10+ 字段；拆成小接口就要
多走十几轮调用，又慢又容易走偏



数据基建 · 友好报错 + 边界约束

友好报错

让 AI 可自主探索

「参数错误」Agent 只能卡住；报错必须告诉它「下一步怎么办」

实例（OOM 分析 Skill 脚本的真实报错）：

✗ 下载成功数不足 (7/10)，
请检查网络或重试 → Agent 自主重试

504/5xx：重试已耗尽，禁止修改请求参数，
直接走回退路径 → 不拿错误参数瞎试

同样是一次报错：一个让 Agent 敢自己重试，
一个让 Agent 立刻换路线——纠偏不需要人参与

边界约束

约束自主探索的风险

先说设计：源码服务对 Agent 暴露的是 bash 接口（git grep / git blame 等，仓库预先聚合索引）——灵活度最大，敞口也最大，所以必须配三道边界：

只读挂载：从根上消除「Agent 改代码」

命令超时：流程可控，不被一条命令占住

完整日志：每一次执行都可追溯

柯南的 Agent 架构 · 端侧基建



端侧基建 · iOS 内存图谱：对齐公域的技术突破

背景：官方 Memory Graph 只在开发环境可用；业界线上方案把采集 + 建图塞进停线程窗口，堆一大就停一两秒

关键技术突破（AI 加持下完成）

两阶段解耦：端侧只采「事实」——原始内存 + 对象识别、
引用关系构建所需元数据；识别建图全部离线，重建标准 memgraph

→ 停顿与对象规模解耦：P50 挂起 ≈300ms、整体采集 ≈4s、无内存增量

端侧性能优化：堆栈压缩存储（ZigZag 编码替代 Radix Tree）

→ 约 10MB 内存即可存下 10 万个对象的分配堆栈，保留分配来源

与官方格式兼容：对齐度 90%+，官方 GUI / heap / vmmap 直接消费

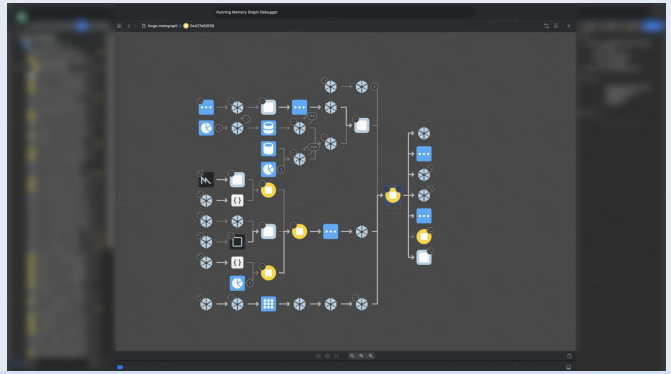
→ 对齐公域，不向模型收「学习税」

≈300ms

线程挂起时长 P50

≈4s 整体采集 · 无内存增量

官方 Memory Graph 工具直接打开自研采集产物



AI 友好 = 与公域对齐

造一套只有自己团队懂的格式，是在向模型收一笔永远收不回来的「学习税」

端侧基建 · Android ANR Trace：把丰富现场搬到线上

ANR 尤其复杂

时间复杂度 · 环境复杂度

→

过去线上采集困难

开销 vs. 完备性两难

→

现在：systrace 线上化

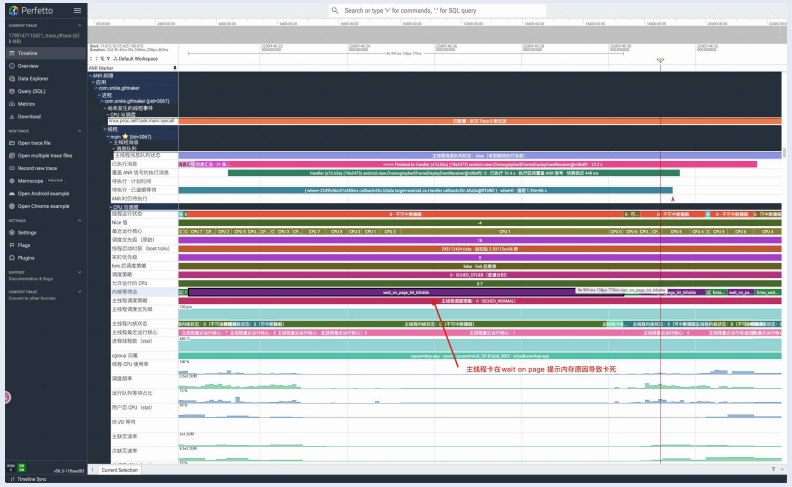
10 大类、200+ 独立数据源

AI 加持下快速开发

Native 下沉 + 共享内存，性能优化到线上可用

采回来的数据由 AI 消费

柯南直接做多跳推理



新 Trace 产物（Perfetto 全景）



柯南 Trace 分析结论

每一跳都需要不同的数据源——

少任何一环，推理就断在那里

这次真实 case：主线程卡在等待换页完成，柯南从 Trace 里逐跳还原到低端机换页 I/O 等待

小结：重上下文建设，靠反馈长出来

① 总判断 · 轻 Agent，重上下文建设

Skills / 数据基建 / 端侧基建，全部投入都在上下文

② 引擎 · 迭代反馈：系统不是建出来的，是长出来的

可观测让失败可回放，人审 + AI 审让每条 case 变成优化项，发现落到哪层就补哪层

③ 经验 · 站在模型一侧做工程：用模型已经懂的方式给上下文

Skills 只教模型不会的、给死停点；端侧格式对齐公域

03

鸿蒙 OOM

治理实战 · 从 ArkTS 到 KMP

为什么是 OOM

鸿蒙 OOM 长期占比 > 50%

现场残缺（堆栈基本没有价值）· 链式累积（累积到临界点才爆发）· 跨语言（泄漏可能发生在任何一层）

OOM 问题主要来自两类：

① 单语言：ArkTS 内存泄漏

对象该释放没释放，在 ArkTS 堆内持续累积

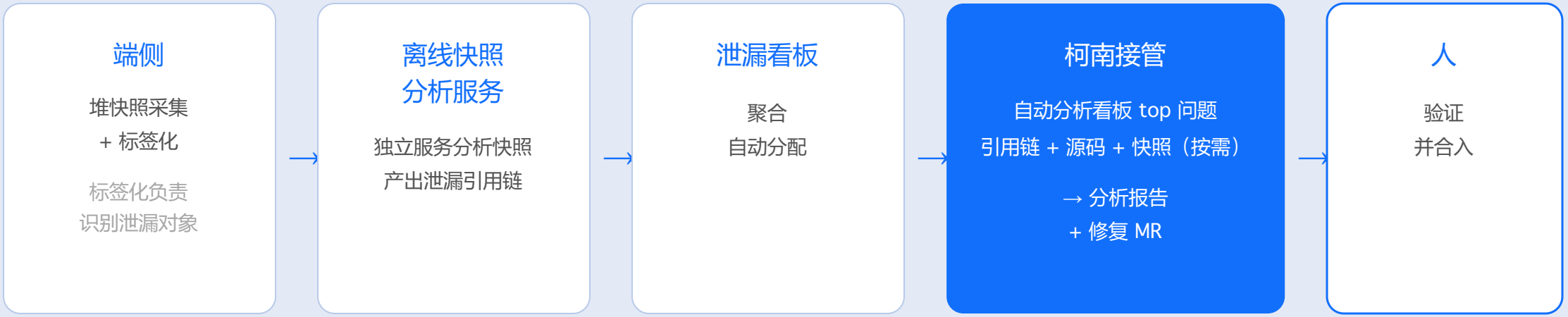
② 跨语言：ArkTS × KMP 循环引用

两个各自正确的 GC，拼出谁都看不见的环

Q3 OOM 指标下降 58%

两周处理 80+ 问题，解决率 80%+

ArkTS 泄漏治理 Loop：整体链路



一条完整的自动化 Loop：从采集到修复 MR，**人只在最后做验证与合入**

ArkTS 泄漏治理 Loop · 关键节点（端侧）

节点一 · 堆快照采集

- 使用 **fork dump** 接口：在 fork 出的子进程里完成堆快照，主进程几乎无感——线上开启的前提

节点二 · 标签化：识别泄漏

- **通用泄漏识别方案：有清晰生命周期的对象都适用——**在结束点（组件 aboutToDisappear / 业务对象释放回调）打「应已销毁」标签；带标签仍 GC 可达 = 确凿泄漏
- 线上可用的关键：避开显式 GC——显式 GC 直接 block 主线程，线上不能做

这一步的关键不是技术，是覆盖度——业务接入越主动，下游 Agent 的分析越准

ArkTS 泄漏治理 Loop · 关键节点（服务化）

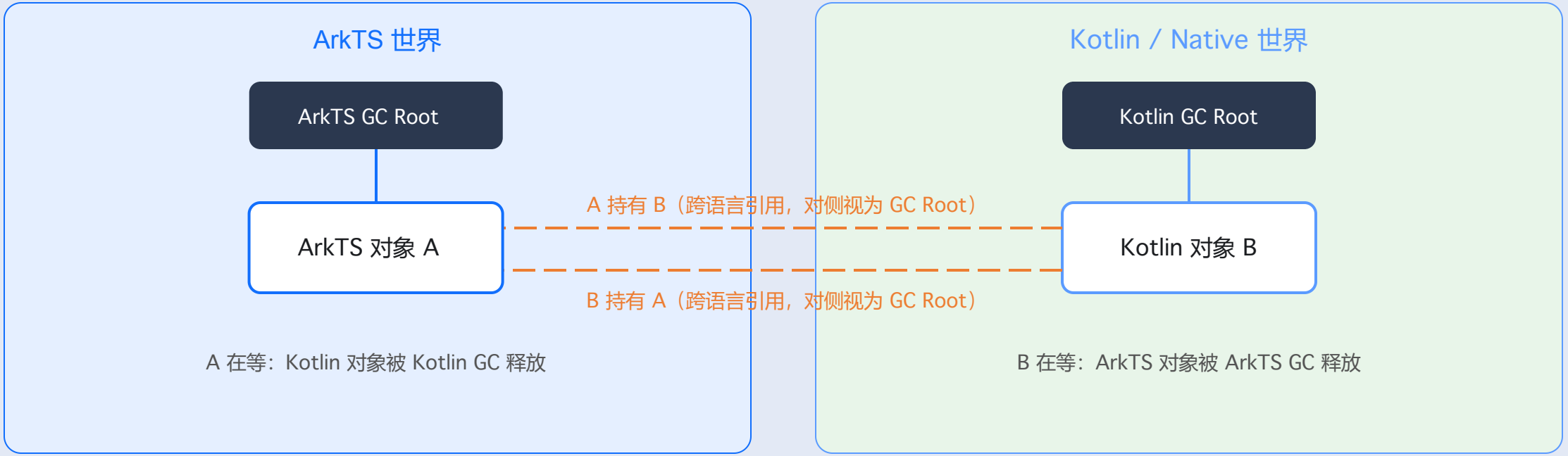
柯南对接由**独立分析服务驱动**，通过 OpenAPI 把「看板 → 柯南 → 人」串成全自动流程



线上运行规模：两周自动修复 60+ 问题

这个量级人已经修不过来了——Loop 的价值是「大力出奇迹」：必须把全流程自动化起来

ArkTS + KMP 循环引用泄漏：为什么会泄漏?



对各自的 GC 来说，本侧对象都被「根」持有着——谁都不释放；双方都在等对端先被释放
单语言视角下每一侧的逻辑都成立，问题只存在于两张对象图拼在一起之后

两个 GC 各自都正确——关键不在「环」，在于跨语言引用被本侧当作看不见的「根」

ArkTS + KMP 循环引用泄漏：端侧基建方案

① 快照采集：双快照，统一 fork dump

ArkTS 侧用 fork dump；Kotlin/Native 侧基于官方 dump 方案扩展支持 fork dump

两份快照在同一时机触发，保证两张图拍的是同一个现场

② 属性关联：基于内部统一 FFI 层（KNAPI），让两侧对象互相「报身份证号」

ArkTS 对象在 Kotlin 侧
= 一个**代理对象（Proxy）**
KNAPI 创建代理时写入对端地址

互认

Kotlin 对象在 ArkTS 侧
= 一个**StableRef**
快照生成时把引用指针写成对象属性

- ArkTS → Kotlin：快照里每个 ArkTS 对象带「我持有哪些 Kotlin 对象」的 StableRef 指针
- Kotlin → ArkTS：每个代理对象自带「我代表哪个 ArkTS 对象」的地址

两侧指针地址双向匹配，服务端即可把两张对象图拼成一张完整的跨语言对象图

ArkTS + KMP 循环引用泄漏：柯南怎么分析

柯南的分析方式：公域知识打底 + 私域知识注入

- heapsnapshot 格式是公域知识；Kotlin kdump 可转 hprof，也是公域知识——模型本来就看得懂
- 唯一需要注入的私域知识：**两侧对象如何相互识别**（上一页的指针属性约定）

→ Agent 直接基于两份快照**生成完整的跨语言引用拓扑图**，并结合源码给出修复方案

为什么这里必须要 AI

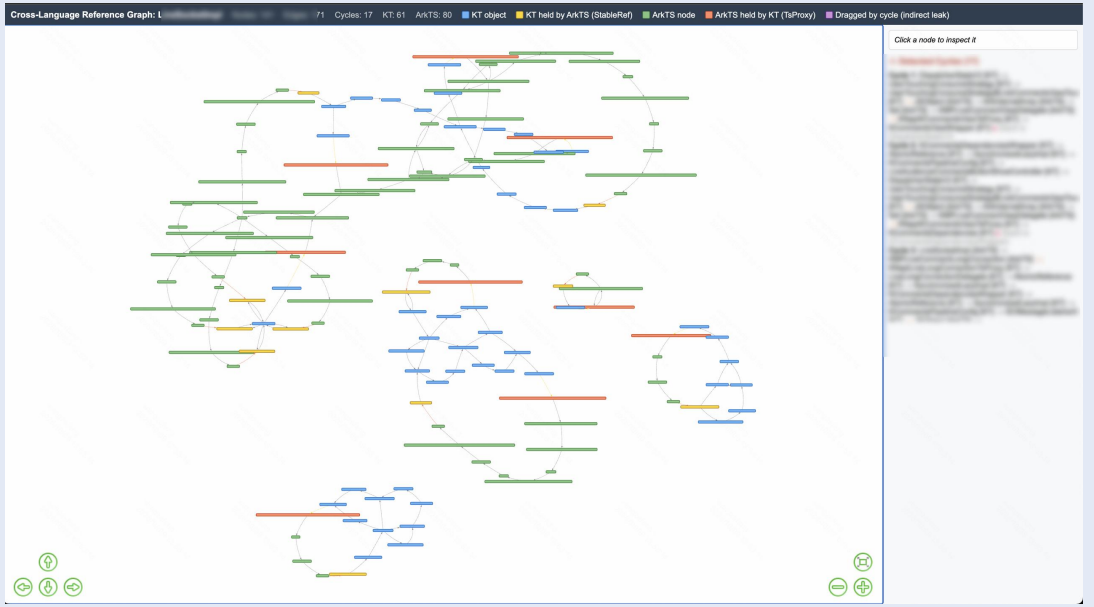
真实环的拓扑复杂度，远超人脑工作记忆

能承载的范围

而且拓扑图最终也不是给人看的——

柯南直接对着它产出修复方案

业务同学只需验收方案、合入



小结：新战场 / 新范式

新战场不是「再做一遍老的事」，
而是「用新范式去做以前做不到的事」。

ArkTS 泄漏：量级大到人修不过来 → 柯南把它变成「采集 → 分析 → 修复 MR」的全自动 Loop

跨语言循环引用：两个 GC 都正确、谁都看不见 → 双快照拼图 + Agent 分析，第一次可发现、可诊断、可修复

共同点：AI 加持下的复杂性已超出人的处理半径——这样的复杂性，要靠 AI 自己来治理

04 总结与展望

从实践回到方法

总结

1. AI 时代的稳定性本质是**吞吐失衡**——处置吞吐必须有一种不依赖人头线性增长的方式跟上

   这个方式，只能是 Agent 自己

2. Agent 是系统工程：

   上下文靠数据基建与端侧基建补齐，工程约束靠结束点定义与私域知识注入落地，效果靠迭代反馈闭环持续提升

   柯南的有效性 = LLM 能力 × 上下文完备度 × Skill 专业性 × 反馈迭代效率

   四个因子相乘，不是相加——缺一个就归零；「换个更强的模型」只提升第一个因子

3. AI 时代一码三端，把复杂性推出人的处理半径

   人学不过来的技术栈，Agent 能学；人顾不过来的跨语言链路，Agent 能串

Agent First, Human in Control

不是把人替换掉，而是让人从跑腿回到判断——AI 制造的复杂性，最终用 AI 来治理

展望

打通验证环节

- 现状瓶颈：Agent 负责「发现问题」，
  人还在负责「验证结果」
- Agent 给出修复方案后，自主完成实际验证
- 验证结果反馈回 Agent，形成
  「分析 → 修复 → 验证 → 沉淀」完整闭环

扩展泛稳定性场景

- 柯南今天主要服务异常退出场景；
  更多问题散落在各业务功能链路——同样缺经验
- 下一步：业务把最痛的场景、最熟的经验补进来
  让同一套范式，解决更多人的日常问题

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

用 AI 打败 AI

